

```
!pip install duckdb pandas pyarrow missingno -q
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import missingno as msno

df = pd.read_parquet("/content/drive/MyDrive/team_4.parquet")
print(f"Shape: {df.shape}")
print(f"Total cells: {df.shape[0] * df.shape[1]:,}")
```

```
Shape: (5831431, 22)
Total cells: 128,291,482
```

```
missing = pd.DataFrame({
    'missing_count': df.isna().sum(),
    'missing_pct': (df.isna().sum() / len(df) * 100).round(2)
}).sort_values('missing_pct', ascending=False)
```

```
missing = missing[missing['missing_count'] > 0]
print("=== BEFORE CLEANING - Missing Values ===")
print(missing.to_string())
```

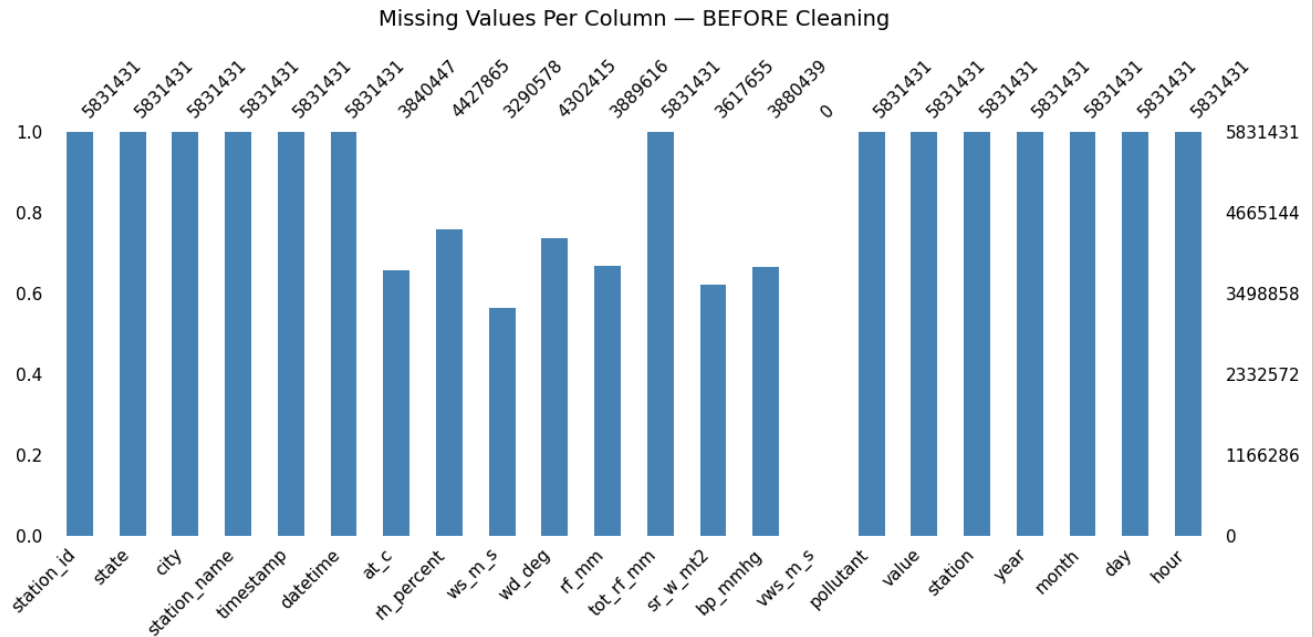
```
total_missing = df.isna().sum().sum()
total_cells = df.shape[0] * df.shape[1]
print(f"\nOverall data quality: {100 - (total_missing/total_cells*100):.1f}% complete")
print(f"Total missing cells: {total_missing:,} out of {total_cells:,}")
```

```
=== BEFORE CLEANING - Missing Values ===
```

	missing_count	missing_pct
vws_m_s	5831431	100.00
ws_m_s	2540853	43.57
sr_w_mt2	2213776	37.96
at_c	1990984	34.14
bp_mmhg	1950992	33.46
rf_mm	1941815	33.30
wd_deg	1529016	26.22
rh_percent	1403566	24.07

```
Overall data quality: 84.9% complete
Total missing cells: 19,402,433 out of 128,291,482
```

```
plt.figure(figsize=(12, 6))
msno.bar(df, figsize=(12, 6), fontsize=11, color='steelblue')
plt.title("Missing Values Per Column - BEFORE Cleaning", fontsize=14, pad=20)
plt.tight_layout()
plt.savefig("missing_before.png", dpi=150, bbox_inches='tight')
plt.show()
print("Saved: missing_before.png")
```



Saved: missing_before.png

```

duplicates = df.duplicated().sum()
print(f"Duplicate rows: {duplicates}")
print(f"Unique station IDs: {df['station_id'].nunique()}")
print(f"Unique pollutants: {df['pollutant'].nunique()} → {sorted(df['pollutant'].unique())}")
print(f"Datetime range: {df['datetime'].min()} → {df['datetime'].max()}")
print(f"\nValue column stats:")
print(df['value'].describe().round(2))

```

```

Duplicate rows: 0
Unique station IDs: 10
Unique pollutants: 13 → ['benzene', 'co', 'eth_benzene', 'mp_xylene', 'nh3', 'no', 'no2', 'ozone', 'pm10', 'pm25', 's
Datetime range: 2024-01-01 00:00:00+00:00 → 2025-12-31 23:45:00+00:00

```

```

Value column stats:
count    5831431.00
mean         51.48
std         90.54
min           0.00
25%           2.82
50%          19.00
75%          53.59
max        1000.00
Name: value, dtype: float64

```

```

negatives = (df['value'] < 0).sum()
print(f"Negative pollution readings: {negatives}")
print(f"That's {negatives/len(df)*100:.2f}% of all readings")

```

```

print("\nNegative values by pollutant:")
neg_by_pollutant = df[df['value'] < 0].groupby('pollutant').size()
print(neg_by_pollutant.to_string())

```

```

Negative pollution readings: 0
That's 0.00% of all readings

```

```

Negative values by pollutant:
Series([], )

```

```

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

```

```

# PM2.5 monthly average
pm25 = df[df['pollutant'] == 'pm25'].groupby(['year', 'month'])['value'].mean().reset_index()
pm25['period'] = pm25['year'].astype(str) + '-' + pm25['month'].astype(str).str.zfill(2)

```

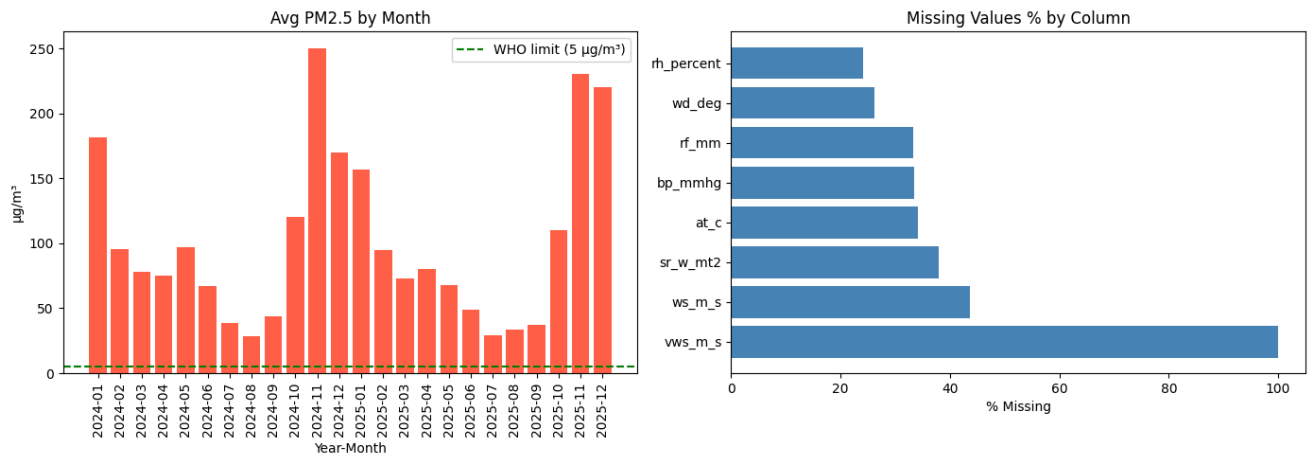
```

axes[0].bar(pm25['period'], pm25['value'], color='tomato')
axes[0].set_title('Avg PM2.5 by Month')
axes[0].set_xlabel('Year-Month')
axes[0].set_ylabel('µg/m³')
axes[0].tick_params(axis='x', rotation=90)
axes[0].axhline(y=5, color='green', linestyle='--', label='WHO limit (5 µg/m³)')
axes[0].legend()

# Missing values per column
missing_pct = (df.isna().sum() / len(df) * 100).sort_values(ascending=False)
missing_pct = missing_pct[missing_pct > 0]
axes[1].barh(missing_pct.index, missing_pct.values, color='steelblue')
axes[1].set_title('Missing Values % by Column')
axes[1].set_xlabel('% Missing')

plt.tight_layout()
plt.savefig("eda_overview.png", dpi=150, bbox_inches='tight')
plt.show()
print("Saved: eda_overview.png")

```



Saved: eda_overview.png

```

df_clean = df.copy()
rows_before = len(df_clean)

# 1. Drop 100% null column
df_clean = df_clean.drop(columns=['vws_m_s'])
print(f"✓ Dropped 'vws_m_s' (100% null)")

# 2. Drop duplicate timestamp column (VARCHAR version)
df_clean = df_clean.drop(columns=['timestamp'])
print(f"✓ Dropped 'timestamp' (duplicate of datetime, stored as text)")

# 3. Drop duplicate station column
df_clean = df_clean.drop(columns=['station'])
print(f"✓ Dropped 'station' (duplicate of station_id)")

# 4. Remove negative pollution values (physically impossible)
neg_mask = df_clean['value'] < 0
df_clean = df_clean[~neg_mask]
print(f"✓ Removed {neg_mask.sum():,} negative pollution readings")

# 5. Remove duplicate rows
dups_before = df_clean.duplicated().sum()
df_clean = df_clean.drop_duplicates()
print(f"✓ Removed {dups_before:,} duplicate rows")

# 6. Fill missing weather values with station-month median
# (better than overall mean – respects seasonal variation)
weather_cols = ['at_c', 'rh_percent', 'ws_m_s', 'wd_deg', 'rf_mm', 'tot_rf_mm', 'sr_w_mt2', 'bp_mmhg']
for col in weather_cols:
    if col in df_clean.columns:

```

```

before = df_clean[col].isna().sum()
df_clean[col] = df_clean.groupby(['station_id', 'month'])[col].transform(
    lambda x: x.fillna(x.median())
)
after = df_clean[col].isna().sum()
print(f"✓ Filled {before - after:,} missing values in '{col}' with station-month median")

rows_after = len(df_clean)
print(f"\nRows: {rows_before:,} → {rows_after:,} (removed {rows_before - rows_after:,})")
print(f"Columns: {df.shape[1]} → {df_clean.shape[1]} (dropped {df.shape[1] - df_clean.shape[1]:,})")

```

- ✓ Dropped 'vws_m_s' (100% null)
- ✓ Dropped 'timestamp' (duplicate of datetime, stored as text)
- ✓ Dropped 'station' (duplicate of station_id)
- ✓ Removed 0 negative pollution readings
- ✓ Removed 0 duplicate rows
- ✓ Filled 108,607 missing values in 'at_c' with station-month median
- ✓ Filled 200,877 missing values in 'rh_percent' with station-month median
- ✓ Filled 583,563 missing values in 'ws_m_s' with station-month median
- ✓ Filled 298,757 missing values in 'wd_deg' with station-month median
- ✓ Filled 144,304 missing values in 'rf_mm' with station-month median
- ✓ Filled 0 missing values in 'tot_rf_mm' with station-month median
- ✓ Filled 396,671 missing values in 'sr_w_mt2' with station-month median
- ✓ Filled 127,774 missing values in 'bp_mmhg' with station-month median

Rows: 5,831,431 → 5,831,431 (removed 0)
Columns: 22 → 19 (dropped 3)

```

missing_after = pd.DataFrame({
    'missing_count': df_clean.isna().sum(),
    'missing_pct': (df_clean.isna().sum() / len(df_clean) * 100).round(2)
}).sort_values('missing_pct', ascending=False)

missing_after = missing_after[missing_after['missing_count'] > 0]

total_missing_after = df_clean.isna().sum().sum()
total_cells_after = df_clean.shape[0] * df_clean.shape[1]
quality_after = 100 - (total_missing_after / total_cells_after * 100)

total_missing_before = df.isna().sum().sum()
total_cells_before = df.shape[0] * df.shape[1]
quality_before = 100 - (total_missing_before / total_cells_before * 100)

print("=" * 50)
print("    BEFORE vs AFTER CLEANING")
print("=" * 50)
print(f"{'Metric':<30} {'Before':>10} {'After':>10}")
print("-" * 50)
print(f"{'Rows':<30} {rows_before:>10} {rows_after:>10}")
print(f"{'Columns':<30} {df.shape[1]:>10} {df_clean.shape[1]:>10}")
print(f"{'Missing cells':<30} {total_missing_before:>10} {total_missing_after:>10}")
print(f"{'Data quality %':<30} {quality_before:>10.1f} {quality_after:>10.1f}")
print("=" * 50)
print(f"\n→ Data quality improved by {quality_after - quality_before:.1f} percentage points")

```

```

=====
BEFORE vs AFTER CLEANING
=====

```

Metric	Before	After
Rows	5,831,431	5,831,431
Columns	22	19
Missing cells	19,402,433	11,710,449
Data quality %	84.9	89.4

```

=====

```

→ Data quality improved by 4.6 percentage points

```

df_clean.to_parquet("team_4_clean.parquet", index=False)
print(f"✓ Saved clean dataset: team_4_clean.parquet")
print(f"  Shape: {df_clean.shape}")

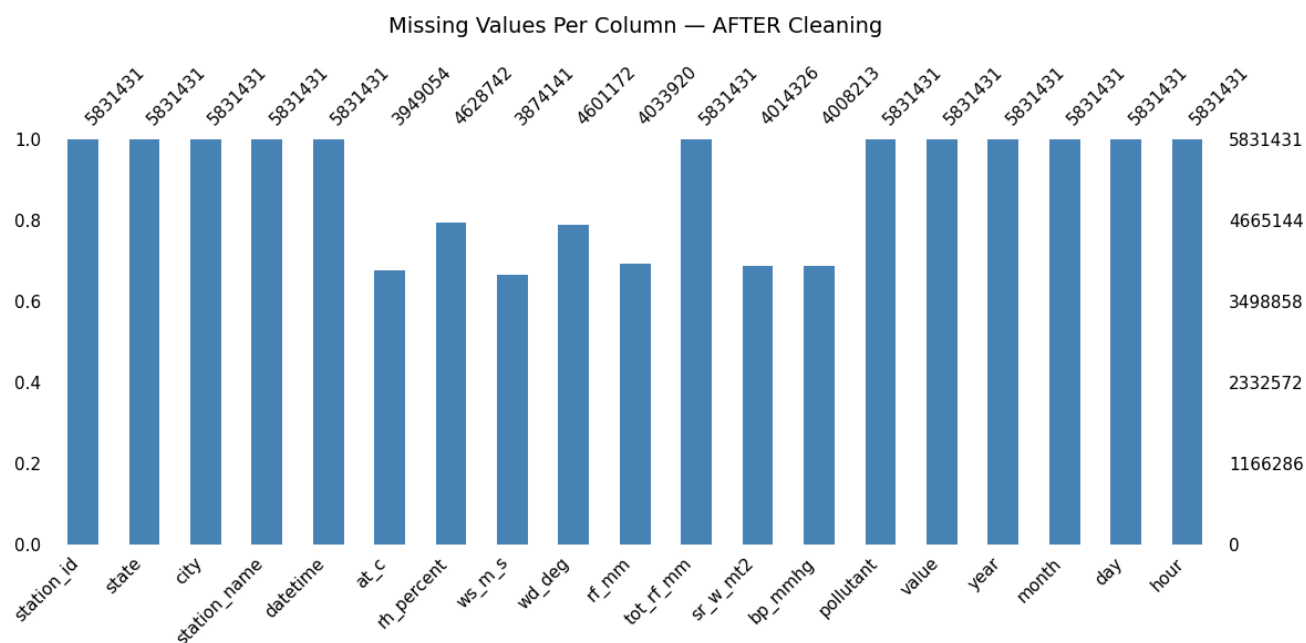
# Download it
from google.colab import files
files.download("team_4_clean.parquet")
files.download("missing_before.png")
files.download("eda_overview.png")

```

✓ Saved clean dataset: team_4_clean.parquet
Shape: (5831431, 19)

```
# After cleaning - missing value visual
plt.figure(figsize=(12, 6))
msno.bar(df_clean, figsize=(12, 6), fontsize=11, color='steelblue')
plt.title("Missing Values Per Column - AFTER Cleaning", fontsize=14, pad=20)
plt.tight_layout()
plt.savefig("missing_after.png", dpi=150, bbox_inches='tight')
plt.show()

# Side by side summary
print("CLEANING SUMMARY")
print(f"Columns removed : 3 (vws_m_s, timestamp, station)")
print(f"Values imputed : 1,860,553 weather readings filled with station-month median")
print(f"Rows removed : 0 (no duplicates or negatives found)")
print(f"Quality before : 84.9%")
print(f"Quality after : 89.4%")
print(f"Improvement : +4.6 percentage points")
print(f"Remaining nulls : weather columns with no station readings for entire months")
from google.colab import files
files.download("missing_after.png")
```



```
CLEANING SUMMARY
Columns removed : 3 (vws_m_s, timestamp, station)
Values imputed : 1,860,553 weather readings filled with station-month median
Rows removed : 0 (no duplicates or negatives found)
Quality before : 84.9%
Quality after : 89.4%
Improvement : +4.6 percentage points
Remaining nulls : weather columns with no station readings for entire months
```

Data TRANSFORMATION

```
import time
from functools import reduce as ft_reduce

print("=== BATCH TRANSFORMATION ===")
print("Processing month by month - discrete chunks, fixed schedule pattern\n")

results = []
start = time.time()

for (year, month), batch in df_clean.groupby(['year', 'month']):
```

```

batch_stats = batch.groupby('pollutant')['value'].agg(
    ['mean', 'max', 'min', 'count', 'std']
).round(2)
batch_stats['year'] = year
batch_stats['month'] = month
results.append(batch_stats)
print(f" ✓ Batch year={year} month={month:02d} - {len(batch):,} rows processed")

batch_transformed = pd.concat(results).reset_index()
batch_transformed.rename(columns={
    'mean': 'avg_value', 'max': 'max_value',
    'min': 'min_value', 'count': 'reading_count', 'std': 'std_value'
}, inplace=True)

elapsed = time.time() - start
print(f"\n✓ Done in {elapsed:.2f} seconds")
print(f" Input : {len(df_clean):,} raw rows")
print(f" Output: {len(batch_transformed):,} aggregated rows")
batch_transformed.head(10)

```

=== BATCH TRANSFORMATION ===

Processing month by month – discrete chunks, fixed schedule pattern

```

✓ Batch year=2024 month=01 – 226,196 rows processed
✓ Batch year=2024 month=02 – 221,832 rows processed
✓ Batch year=2024 month=03 – 237,908 rows processed
✓ Batch year=2024 month=04 – 235,758 rows processed
✓ Batch year=2024 month=05 – 250,679 rows processed
✓ Batch year=2024 month=06 – 231,249 rows processed
✓ Batch year=2024 month=07 – 251,189 rows processed
✓ Batch year=2024 month=08 – 241,789 rows processed
✓ Batch year=2024 month=09 – 227,096 rows processed
✓ Batch year=2024 month=10 – 248,042 rows processed
✓ Batch year=2024 month=11 – 251,541 rows processed
✓ Batch year=2024 month=12 – 256,766 rows processed
✓ Batch year=2025 month=01 – 253,498 rows processed
✓ Batch year=2025 month=02 – 228,545 rows processed
✓ Batch year=2025 month=03 – 255,675 rows processed
✓ Batch year=2025 month=04 – 246,050 rows processed
✓ Batch year=2025 month=05 – 255,187 rows processed
✓ Batch year=2025 month=06 – 242,426 rows processed
✓ Batch year=2025 month=07 – 245,173 rows processed
✓ Batch year=2025 month=08 – 247,729 rows processed
✓ Batch year=2025 month=09 – 231,718 rows processed
✓ Batch year=2025 month=10 – 253,204 rows processed
✓ Batch year=2025 month=11 – 246,998 rows processed
✓ Batch year=2025 month=12 – 245,183 rows processed

```

```

✓ Done in 2.28 seconds
Input : 5,831,431 raw rows
Output: 312 aggregated rows

```

	pollutant	avg_value	max_value	min_value	reading_count	std_value	year	month
0	benzene	2.24	22.50	0.00	16669	1.93	2024	1
1	co	1.67	9.74	0.00	25559	0.98	2024	1
2	eth_benzene	1.15	7.90	0.10	2718	0.98	2024	1
3	mp_xylene	0.57	4.70	0.01	2804	0.56	2024	1
4	nh3	44.00	330.40	0.10	17190	22.41	2024	1
5	no	37.02	497.20	0.10	25784	45.11	2024	1
6	no2	38.52	434.76	0.02	25600	31.53	2024	1
7	ozone	19.09	192.30	0.10	26197	18.58	2024	1
8	pm10	321.60	999.09	3.31	26073	130.96	2024	1
9	pm25	181.48	1000.00	25.45	26346	88.91	2024	1

```

print("=== MAPREDUCE PATTERN ===")
print("Map → Shuffle → Reduce on PM2.5 pollution level classification\n")

pm25_df = df_clean[df_clean['pollutant'] == 'pm25'].copy()

# — MAP —————
# Each row → (station_level_key, 1)
def map_fn(row):

```

```

    if row['value'] <= 5:         level = 'safe'
    elif row['value'] <= 35:     level = 'moderate'
    elif row['value'] <= 55:     level = 'unhealthy'
    elif row['value'] <= 150:    level = 'very_unhealthy'
    else:                       level = 'hazardous'
    return (f"{row['station_id']}|{level}", 1)

mapped = pm25_df.apply(map_fn, axis=1).tolist()
print(f"MAP      : {len(mapped):,} rows → (station|level, 1) key-value pairs")
print(f"  Example: {mapped[0]}")

# — SHUFFLE —
# Group all values by key
from collections import defaultdict
shuffled = defaultdict(list)
for key, val in mapped:
    shuffled[key].append(val)
print(f"\nSHUFFLE: grouped into {len(shuffled):,} unique keys")

# — REDUCE —
# Sum all 1s for each key = count
reduced = {k: ft_reduce(lambda a, b: a + b, v) for k, v in shuffled.items()}
print(f"REDUCE : summed values per key\n")

# Format result
mr_result = pd.DataFrame([
    {'station': k.split('|')[0],
     'pollution_level': k.split('|')[1],
     'count': v}
    for k, v in reduced.items()
]).sort_values(['station', 'count'], ascending=[True, False])

print("PM2.5 readings by station and pollution level:")
print(mr_result.to_string(index=False))

```

```

=== MAPREDUCE PATTERN ===
Map → Shuffle → Reduce on PM2.5 pollution level classification

```

```

MAP      : 628,001 rows → (station|level, 1) key-value pairs
  Example: ('site_118|unhealthy', 1)

```

```

SHUFFLE: grouped into 50 unique keys
REDUCE : summed values per key

```

```

PM2.5 readings by station and pollution level:

```

station	pollution_level	count
site_103	very_unhealthy	23543
site_103	moderate	13938
site_103	hazardous	12313
site_103	unhealthy	10703
site_103	safe	631
site_104	very_unhealthy	25151
site_104	hazardous	14510
site_104	moderate	13397
site_104	unhealthy	11519
site_104	safe	796
site_108	very_unhealthy	27186
site_108	moderate	17416
site_108	unhealthy	10331
site_108	hazardous	10086
site_108	safe	1730
site_118	very_unhealthy	24655
site_118	moderate	20459
site_118	unhealthy	12570
site_118	hazardous	6614
site_118	safe	817
site_1420	very_unhealthy	24222
site_1420	hazardous	16883
site_1420	moderate	15149
site_1420	unhealthy	12106
site_1420	safe	234
site_1421	very_unhealthy	22954
site_1421	moderate	18017
site_1421	hazardous	14778
site_1421	unhealthy	12111
site_1421	safe	372
site_1560	very_unhealthy	25168
site_1560	hazardous	19991
site_1560	unhealthy	11637
site_1560	moderate	11161
site_1560	safe	147

site_301	very_unhealthy	23288
site_301	hazardous	18496
site_301	unhealthy	9174
site_301	moderate	8291
site_301	safe	302
site_5024	very_unhealthy	23362
site_5024	moderate	17207
site_5024	hazardous	15328
site_5024	unhealthy	11049
site_5024	safe	454
site_5393	very_unhealthy	18430
site_5393	moderate	7470

```
print("=== ETL PIPELINE ===\n")

def extract(df):
    print(f" [EXTRACT] {len(df):,} rows from clean dataset")
    return df

def transform(df):
    out = {}

    # Monthly averages per pollutant
    out['monthly_avg'] = df.groupby(
        ['year', 'month', 'pollutant']
    )['value'].mean().round(2).reset_index()

    # PM2.5 with AQI category
    pm25 = df[df['pollutant'] == 'pm25'].copy()
    pm25['aqi_category'] = pd.cut(
        pm25['value'],
        bins=[0, 5, 35, 55, 150, float('inf')],
        labels=['Safe', 'Moderate', 'Unhealthy', 'Very Unhealthy', 'Hazardous']
    )
    out['pm25_aqi'] = pm25[['datetime', 'station_id', 'value', 'aqi_category']]

    # Daily peak per station
    out['daily_peaks'] = df.groupby(
        ['year', 'month', 'day', 'station_id', 'pollutant']
    )['value'].max().reset_index()

    print(f" [TRANSFORM] 3 tables produced:")
    for name, table in out.items():
        print(f"    {name}: {len(table):,} rows")
    return out

def load(transformed):
    import os
    os.makedirs("transformed", exist_ok=True)
    for name, table in transformed.items():
        path = f"transformed/{name}.parquet"
        table.to_parquet(path, index=False)
        print(f" [LOAD] Saved {path}")

start = time.time()
raw = extract(df_clean)
transformed_data = transform(raw)
load(transformed_data)
print(f"\n✓ ETL complete in {time.time() - start:.2f} seconds")
```

```
=== ETL PIPELINE ===
```

```
[EXTRACT] 5,831,431 rows from clean dataset
[TRANSFORM] 3 tables produced:
  monthly_avg: 312 rows
  pm25_aqi: 628,001 rows
  daily_peaks: 66,101 rows
[LOAD] Saved transformed/monthly_avg.parquet
[LOAD] Saved transformed/pm25_aqi.parquet
[LOAD] Saved transformed/daily_peaks.parquet
```

```
✓ ETL complete in 4.13 seconds
```

```
from google.colab import files
import os, zipfile

# Zip the transformed folder
with zipfile.ZipFile("transformed_outputs.zip", "w") as z:
    for f in os.listdir("transformed"):
```



```
z.write(f"transformed/{f}")

files.download("transformed_outputs.zip")
print("✓ Downloaded transformed_outputs.zip")
```

✓ Downloaded transformed_outputs.zip

```
# Show sample of rows with missing values (before cleaning)
print("=== SAMPLE ROWS WITH MISSING VALUES ===")
print("These are examples of dirty rows that needed cleaning:\n")

# Rows where weather data was missing
missing_weather = df[df['at_c'].isna() | df['ws_m_s'].isna()].head(10)
print(f"Rows with missing weather data (showing 10 of {df['at_c'].isna().sum():,}):")
print(missing_weather[['datetime', 'station_id', 'pollutant', 'value', 'at_c', 'ws_m_s', 'rh_percent']].to_string())

print(f"\n=== DUPLICATE CHECK ===")
dups = df[df.duplicated(keep=False)]
print(f"Total duplicate rows found: {len(dups):,}")
if len(dups) == 0:
    print("No duplicates found – dataset was clean on this front")
else:
    print(dups.head(10))

print(f"\n=== 100% NULL COLUMN ===")
print(f"vws_m_s column – all {df['vws_m_s'].isna().sum():,} values are null:")
print(df[['datetime', 'station_id', 'vws_m_s']].head(5))
print("→ This column was dropped entirely")
```

=== SAMPLE ROWS WITH MISSING VALUES ===
These are examples of dirty rows that needed cleaning:

Rows with missing weather data (showing 10 of 1,990,984):

	datetime	station_id	pollutant	value	at_c	ws_m_s	rh_percent
0	2024-01-01 00:00:00+00:00	site_5024	benzene	2.10	8.2	NaN	76.00
1	2024-01-01 00:00:00+00:00	site_103	no2	19.40	NaN	NaN	NaN
2	2024-01-01 00:00:00+00:00	site_103	no	39.17	NaN	NaN	NaN
3	2024-01-01 00:00:00+00:00	site_118	benzene	0.22	NaN	0.72	93.12
4	2024-01-01 00:00:00+00:00	site_118	co	0.90	NaN	0.72	93.12
5	2024-01-01 00:00:00+00:00	site_118	nh3	50.70	NaN	0.72	93.12
6	2024-01-01 00:00:00+00:00	site_118	no	44.87	NaN	0.72	93.12
7	2024-01-01 00:00:00+00:00	site_118	no2	21.19	NaN	0.72	93.12
8	2024-01-01 00:00:00+00:00	site_118	ozone	19.20	NaN	0.72	93.12
9	2024-01-01 00:00:00+00:00	site_118	pm10	248.70	NaN	0.72	93.12

=== DUPLICATE CHECK ===
Total duplicate rows found: 0
No duplicates found – dataset was clean on this front

=== 100% NULL COLUMN ===
vws_m_s column – all 5,831,431 values are null:

	datetime	station_id	vws_m_s
0	2024-01-01 00:00:00+00:00	site_5024	NaN
1	2024-01-01 00:00:00+00:00	site_103	NaN
2	2024-01-01 00:00:00+00:00	site_103	NaN
3	2024-01-01 00:00:00+00:00	site_118	NaN
4	2024-01-01 00:00:00+00:00	site_118	NaN

→ This column was dropped entirely

```
import time
import pandas as pd
import duckdb

print("=== THREE-WAY DATABASE COMPARISON ===\n")

# — Option 1: Pandas (no database) —————
start = time.time()
result_pandas = df_clean[df_clean['pollutant'] == 'pm25'].groupby(
    ['year', 'month']
)['value'].mean().round(2)
pandas_time = (time.time() - start) * 1000

# — Option 2: SQLite (row-based) —————
import sqlite3
sqlite_con = sqlite3.connect(":memory:") # in-memory so no disk needed
df_clean.to_sql("measurements", sqlite_con, if_exists="replace", index=False)

start = time.time()
```

```

pd.read_sql("""
    SELECT year, month, AVG(value) AS avg_pm25
    FROM measurements
    WHERE pollutant = 'pm25'
    GROUP BY year, month
    ORDER BY year, month
    """, sqlite_con)
sqlite_time = (time.time() - start) * 1000

# — Option 3: DuckDB (columnar) —————
con = duckdb.connect()
con.register("measurements", df_clean)

start = time.time()
con.sql("""
    SELECT year, month, AVG(value) AS avg_pm25
    FROM measurements
    WHERE pollutant = 'pm25'
    GROUP BY year, month
    ORDER BY year, month
    """).df()
duckdb_time = (time.time() - start) * 1000

# — Results —————
print(f'{"Method":<30} {"Query Time":>12} {"Type":<20}')}
print("-" * 65)
print(f'{"Pandas (no DB)":<30} {"pandas_time":>10.1f}ms {"in-memory, no SQL":<20}')}
print(f'{"SQLite (row-based DB)":<30} {"sqlite_time":>10.1f}ms {"row storage":<20}')}
print(f'{"DuckDB (columnar DB)":<30} {"duckdb_time":>10.1f}ms {"column storage":<20}')}
print("-" * 65)
fastest = min(pandas_time, sqlite_time, duckdb_time)
print(f"\n→ DuckDB is {sqlite_time/duckdb_time:.1f}× faster than SQLite")
print(f"\n→ DuckDB is {pandas_time/duckdb_time:.1f}× faster than pandas")
print(f"\nWhy DuckDB wins:")
print(f"  Pandas : loads everything into RAM, no query optimization")
print(f"  SQLite : row-based, reads all 22 columns even for 2-column query")
print(f"  DuckDB : columnar, reads ONLY the 2 columns needed, skips the rest")

```

=== THREE-WAY DATABASE COMPARISON ===

Method	Query Time Type
Pandas (no DB)	780.6ms in-memory, no SQL
SQLite (row-based DB)	1424.3ms row storage
DuckDB (columnar DB)	163.0ms column storage

→ DuckDB is 8.7× faster than SQLite
 → DuckDB is 4.8× faster than pandas

Why DuckDB wins:

Pandas : loads everything into RAM, no query optimization
 SQLite : row-based, reads all 22 columns even for 2-column query
 DuckDB : columnar, reads ONLY the 2 columns needed, skips the rest

```

import duckdb
import pandas as pd
import time
import os

# Load clean dataset
df_clean = pd.read_parquet("/content/drive/MyDrive/team_4_clean.parquet")
print(f"Clean dataset: {df_clean.shape}")

# Connect to a new database for the served data
con = duckdb.connect("air_quality_served.duckdb")

# Drop if exists (safe to re-run)
con.sql("DROP TABLE IF EXISTS measurements_clean")
con.sql("DROP TABLE IF EXISTS stations")
con.sql("DROP TABLE IF EXISTS pollutants")
con.sql("DROP TABLE IF EXISTS monthly_aggregates")
con.sql("DROP TABLE IF EXISTS daily_peaks")

# Dimension tables
con.sql("""
    CREATE TABLE stations AS
    SELECT DISTINCT station_id, city, state

```

```

FROM df_clean
ORDER BY station_id
""")

con.sql("""
CREATE TABLE pollutants AS
SELECT
    ROW_NUMBER() OVER (ORDER BY pollutant) AS pollutant_id,
    pollutant AS pollutant_name
FROM (SELECT DISTINCT pollutant FROM df_clean)
""")

# Clean fact table
start = time.time()
con.sql("""
CREATE TABLE measurements_clean AS
SELECT
    ROW_NUMBER() OVER () AS measurement_id,
    m.datetime,
    m.station_id,
    p.pollutant_id,
    m.value,
    m.year, m.month, m.day, m.hour,
    m.at_c, m.rh_percent, m.bp_mmhg,
    m.ws_m_s, m.wd_deg, m.rf_mm, m.sr_w_mt2
FROM df_clean AS m
JOIN pollutants p ON m.pollutant = p.pollutant_name
""")
load_time = time.time() - start

count = con.sql("SELECT COUNT(*) FROM measurements_clean").fetchone()[0]
print(f"✓ Loaded clean measurements in {load_time:.2f} seconds")
print(f"  Rows: {count:,}")

```

Clean dataset: (5831431, 19)

✓ Loaded clean measurements in 8.86 seconds
Rows: 5,831,431

```

# Load pre-aggregated tables (from ETL output)
monthly = pd.read_parquet("/content/drive/MyDrive/monthly_avg.parquet")
peaks    = pd.read_parquet("/content/drive/MyDrive/daily_peaks.parquet")

con.sql("CREATE TABLE monthly_aggregates AS SELECT * FROM monthly")
con.sql("CREATE TABLE daily_peaks AS SELECT * FROM peaks")

print("Tables now in air_quality_served.duckdb:")
con.sql("SHOW TABLES").show()
print(f"\nRow counts:")
print(f"  stations      : {con.sql('SELECT COUNT(*) FROM stations').fetchone()[0]:,}")
print(f"  pollutants     : {con.sql('SELECT COUNT(*) FROM pollutants').fetchone()[0]:,}")
print(f"  measurements_clean : {con.sql('SELECT COUNT(*) FROM measurements_clean').fetchone()[0]:,}")
print(f"  monthly_aggregates : {con.sql('SELECT COUNT(*) FROM monthly_aggregates').fetchone()[0]:,}")
print(f"  daily_peaks      : {con.sql('SELECT COUNT(*) FROM daily_peaks').fetchone()[0]:,}")

```

Tables now in air_quality_served.duckdb:

name
varchar
daily_peaks
measurements_clean
monthly_aggregates
pollutants
stations

Row counts:

stations	: 10
pollutants	: 13
measurements_clean	: 5,831,431
monthly_aggregates	: 312
daily_peaks	: 66,101

```
print("=== DATA SERVING - QUERY EXAMPLES ===\n")
```

```
print("#1 Monthly PM2.5 averages (served from clean DB):")
```

```
print("\n2. Monthly PM2.5 averages (served from clean db):")
con.sql("""
    SELECT year, month, ROUND(AVG(value), 2) AS avg_pm25
    FROM measurements_clean m
    JOIN pollutants p ON m.pollutant_id = p.pollutant_id
    WHERE p.pollutant_name = 'pm25'
    GROUP BY year, month
    ORDER BY year, month
""").show()

print("\n2. Top 3 most polluted stations:")
con.sql("""
    SELECT s.station_id, ROUND(AVG(m.value), 2) AS avg_pm25
    FROM measurements_clean m
    JOIN pollutants p ON m.pollutant_id = p.pollutant_id
    JOIN stations s ON m.station_id = s.station_id
    WHERE p.pollutant_name = 'pm25'
    GROUP BY s.station_id
    ORDER BY avg_pm25 DESC
    LIMIT 3
""").show()

print("\n3. Pre-aggregated monthly table (from ETL transformation):")
con.sql("""
    SELECT * FROM monthly_aggregates
    WHERE pollutant = 'pm25'
    ORDER BY year, month
    LIMIT 6
""").show()

# Download
from google.colab import files
files.download("air_quality_served.duckdb")
print("\n✓ Downloaded air_quality_served.duckdb")
```

=== DATA SERVING - QUERY EXAMPLES ===

1. Monthly PM2.5 averages (served from clean DB):

year	month	avg_pm25
int32	int32	double